

AETHER

Environmental Health Intelligence & Air Quality Prediction Platform

Comprehensive Project Documentation — Full Development Journey

Prepared by	Nada Hosny Khalifa
Track	AI & Data Science — Microsoft Data Engineer
Programme	روداد مصر الرقمية — Digital Egypt Pioneers
Location	Cairo, Egypt
Period	2025 – 2026
Document Version	v3.0 — Final

Table of Contents

1. **Executive Summary**
 - 1.1 Delivered Capabilities
 - 1.2 Technology Stack
 - 1.3 Key Metrics at a Glance
2. **Problem Statement**
 - 2.1 Gap Analysis Table
 - 2.2 The AETHER Solution
3. **System Architecture**
 - 3.1 Architectural Layers (V2 — Final Production)
 - 3.2 Medallion Architecture (Bronze / Silver / Gold)
 - 3.3 Data Warehouse Schema — AetherDW_V0
4. **Development Journey — Milestone Overview**
 - Milestone Progress Tracker
 - Milestone Summary Table (all 9 milestones with obstacles count)
5. **4. Development Journey — Detailed Milestones**
 - **Milestone 1 — Environment & Project Setup**
 - 1.1 Objective
 - 1.2 What Was Accomplished
 - 1.3 Obstacles: Python Not Recognised in PowerShell / Two Python Installations Causing Confusion
 - 1.4 Milestone Outcome
 - **Milestone 2 — Data Ingestion**
 - 2.1 Objective
 - 2.2 Data Sources Identified & Integrated
 - 2.3 Python Fetch Scripts Created
 - 2.4 Obstacle: OpenAQ API Returned Empty Results for Cairo
 - 2.5 Milestone Outcome
 - **Milestone 3 — Data Cleaning & Profiling**
 - 3.1 Objective
 - 3.2 Profiling Results
 - 3.3 Cleaning Strategy Applied
 - 3.4 Obstacles: dropna() Silently Destroyed All Weather Data / Date Conversion SQL Server Error
 - 3.5 Milestone Outcome
 - **Milestone 4 — Feature Engineering**
 - 4.1 Objective
 - 4.2 Two-Mode Design (API Mode vs Historical Mode)
 - 4.3 Date-Based Dataset Merge
 - 4.4 Engineered Features (8 features with formulas)
 - 4.5 Health Category Classification — Calibrated for Cairo
 - 4.6 Obstacle: Class Imbalance — 97% of Training Data in One Category
 - 4.7 Milestone Outcome
 - **Milestone 5 — SQL Server Data Warehouse**
 - 5.1 Objective
 - 5.2 Schema Design Decision — Constellation Schema
 - 5.3 Python SQL Loader (Initial Implementation)

- 5.4 Obstacles: SQL Server Connection Error 08001 / NULL Values Inserted into Every Column
 - 5.5 Milestone Outcome
 - **Milestone 6 — SSIS ETL Packages**
 - 6.1 Objective
 - 6.2 Tool Setup
 - 6.3 SSIS Packages Created
 - 6.4 Incremental Loading Design
 - 6.5 Obstacles: Code Page Conflict (1252 vs 1256) / SSIS Package Not Found at Runtime
 - 6.6 Milestone Outcome
 - **Milestone 7 — Machine Learning Model**
 - 7.1 Objective
 - 7.2 Model Architecture & Hyperparameters
 - 7.3 Training Features (13 Total)
 - 7.4 Model Performance Results (per-category F1 scores)
 - 7.5 Feature Importance Analysis (Top 5)
 - 7.6 Obstacle: Trivial 98% Accuracy Due to Class Imbalance
 - 7.7 Milestone Outcome
 - **Milestone 8 — Prefect Orchestration**
 - 8.1 Objective
 - 8.2 Tool Evaluation & Selection (Airflow vs Task Scheduler vs Prefect)
 - 8.3 Pipeline Architecture (5 tasks with retry config)
 - 8.4 Key Engineering Decisions
 - 8.5 Obstacles: ModuleNotFoundError utils.logger / SSIS Package Path Mismatch / Global Python in Subprocess
 - 8.6 Validated Pipeline Execution Log (actual output)
 - 8.7 Milestone Outcome
 - **Milestone 9 — Power BI Dashboard**
 - 9.1 Objective
 - 9.2 Data Model Design & Relationships
 - 9.3 DAX Measures
 - 9.4 Dashboard Pages (Home / Environmental Overview / Health Intelligence / Forecast Analytics / Seasonal Analysis)
 - 9.5 Obstacle: Relationship Ambiguity Across Multiple Fact Tables
 - 9.6 Milestone Outcome
6. **Key Performance Indicators**
- 5.1 Data Engineering KPIs
 - 5.2 SQL Warehouse KPIs
 - 5.3 Machine Learning KPIs
 - 5.4 Visualisation & Automation KPIs
7. **Lessons Learned**
- 6.1 Data Quality Is the Hardest Part
 - 6.2 Execution Environment Matters More Than Code
 - 6.3 Domain Knowledge Shapes Model Quality
 - 6.4 Tool Selection Should Match Real Constraints
 - 6.5 Schema Design Prevents Downstream Problems
 - 6.6 Constellation Over Simple Star Schema
8. **Future Enhancements**

9. **Project Deliverables**

- 8.1 Technical Deliverables
- 8.2 Repository Structure

10. **Conclusion**

1. Executive Summary

AETHER is an end-to-end environmental health intelligence platform developed to address a critical gap in public health awareness: while environmental data is widely collected in Cairo and globally, most monitoring systems present raw measurements without translating them into actionable guidance.

The platform ingests weather and air quality data from both historical archives and live REST APIs, processes it through a fully automated ETL pipeline, stores it in a SQL Server data warehouse, and applies a machine learning model to classify each day into one of five human-readable health risk categories. Insights are delivered through an interactive Power BI dashboard and automated daily via Prefect workflow orchestration.

Delivered Capabilities

1	Automated ETL pipeline fetching live environmental data from two REST APIs daily
2	Historical data integration covering weather (2000–2025) and pollution (2022–2026) for Cairo
3	8 engineered environmental health features derived from raw measurements
4	Cairo-calibrated health classification system with 5 human-readable risk categories
5	Random Forest ML model achieving 97% F1 score after Cairo-specific threshold calibration
6	Multi-day (Day +1 / +2 / +3) environmental forecast generation with confidence tracking
7	SQL Server constellation schema warehouse with SSIS-based incremental loading
8	Interactive multi-page Power BI dashboard with DAX-driven dynamic health guidance
9	Prefect orchestration pipeline completing the full daily workflow in ~21 seconds

Technology Stack

Category	Technology	Purpose
Language	Python 3.x	Data processing, feature engineering, ML, pipeline scripts
Data Processing	Pandas, NumPy	Cleaning, transformation, feature computation
Machine Learning	Scikit-learn (RandomForestClassifier)	Health risk classification model
Database	SQL Server 2022	Data warehouse — AetherDW_V0

ETL Tooling	SSIS (SQL Server Integration Services)	Production-grade incremental data loading
Orchestration	Prefect	Daily pipeline automation, retry logic, logging
Weather API	OpenWeatherMap API	Live daily weather data ingestion
Pollution API	WAQI API (Station A527650)	Live daily air quality ingestion
Visualisation	Power BI Desktop	Interactive environmental health dashboard
Version Control	Git + GitHub	Source control and portfolio
Model Persistence	joblib	Trained model serialisation and reuse

Key Metrics at a Glance

97% ML F1 Score After Cairo calibration	1,154 Training Rows 2022–2025 overlap	21s Full Pipeline Daily execution	9 Milestones Completed	15 Obstacles Resolved
--	--	--	---	--

2. Problem Statement

Air pollution is one of the most serious environmental threats to human health worldwide. According to the World Health Organization, air pollution contributes to approximately 7 million premature deaths globally every year. Over 90% of the world's population breathes air that exceeds WHO recommended limits.

Cairo, Egypt experiences persistently elevated pollution levels driven by high traffic density, industrial activity, and seasonal agricultural burning. Despite the availability of environmental monitoring data, the following fundamental gaps remain unaddressed:

Gap	Description	Impact on Citizens
Raw Data Problem	Environmental data presented as raw AQI numbers and $\mu\text{g}/\text{m}^3$ values is not self-explanatory	Citizens cannot interpret whether a given AQI value represents a real health risk
Isolated Data	Monitoring data exists in isolation with no integration to health guidance systems	No connection between environmental readings and what actions citizens should take
Reactive Systems	Current systems react to past readings rather than generating forward predictions	No early warning capability for upcoming high-risk days
No Targeting	Vulnerable populations (asthma, elderly, children) receive no tailored risk warnings	Highest-risk individuals are not specifically protected
Fragmented Sources	Weather and pollution data exist in separate systems that are never merged	No composite health indicators that combine temperature stress with pollution

The AETHER Solution

MISSION	AETHER was built to bridge this gap by creating a data platform that translates raw environmental measurements into daily health intelligence that any citizen can act on — expressed not as numbers but as clear categories like 'Safe Air Day' or 'Avoid Outdoor Activity Day'.
---------	---

3. System Architecture

AETHER is structured as a layered data platform where each component feeds the next in a clean sequential pipeline. The architecture evolved through two major versions, ultimately arriving at a production-grade constellation schema design with SSIS-based loading and Prefect orchestration.

3.1 Architectural Layers (V2 — Final Production)

Layer	Component	Input	Output
1 — Sources	OpenWeatherMap + WAQI APIs + Historical CSVs	External APIs / CSV files	Raw JSON + raw CSV data
2 — Ingestion	Python fetch scripts	API endpoints	data/raw/weather_api_YYYYMMDD.json, pollution_api_YYYYMMDD.json
3 — Cleaning	data_cleaning.py	Raw CSVs / JSON	data/staging/ — normalised, null-handled CSVs
4 — Features	feature_engineering.py	Cleaned staging data	data/processed/environmental_features.csv
5 — Loading	SSIS packages (dtexec)	environmental_features.csv	SQL Server AetherDW_V0 tables
6 — ML Prediction	predict_risk.py (Random Forest)	EnvironmentalFeatures	Gold.RiskPredictions
7 — Forecasting	forecast pipeline	Current conditions + model	Gold.ForecastPredictions (Day +1/+2/+3)
8 — Automation	Prefect flow	All scripts	Orchestrated daily execution with retry + logging
9 — Visualisation	Power BI Desktop	SQL Server tables	Interactive health intelligence dashboard

3.2 Medallion Architecture

Zone	Schema	Purpose	Primary Table
Bronze	Bronze	Raw data landing zone — immutable, source-faithful storage	Raw CSVs, API JSON files
Silver	Silver	Cleaned, normalised, transformed and merged datasets	Cleaned weather + pollution CSVs
Gold	Gold	Analytics-ready curated layer for ML, BI, and forecasting	Gold.EnvironmentalFeatures

3.3 Data Warehouse Schema — AetherDW_V0

A constellation schema (galaxy schema) was chosen over a simple star schema because AETHER has multiple independent fact tables all relating to a single date dimension. This is the standard approach in production analytics platforms.

Table	Schema Type	Key Columns	Role
DimDate	Dimension	date, year, month, quarter, week, is_weekend	Central time dimension — one row per date, governs all filtering
WeatherMetrics	Fact	date, temperature, humidity, wind, pressure, cloud_cover	Raw weather readings with source tracking
PollutionMetrics	Fact	date, pm25, pm10, aqi, ozone, nitrogen_dioxide, sulphur_dioxide	Raw pollution readings
EnvironmentalFeatures	Fact (Gold)	date, heat_index, pollution_level, respiratory_stress, uv_risk, health_category	Merged + enriched table — primary analytics source
RiskPredictions	Fact	date, health_category, model_version, predicted_at	ML model daily output
ForecastPredictions	Fact	forecast_date, forecast_horizon, predicted_category, confidence	Multi-day forecast outputs

4. Development Journey — Milestone Overview

The AETHER project was developed across 9 structured milestones, each building on the previous. Every milestone encountered real-world technical obstacles that required engineering decisions and targeted solutions. This section documents the complete development journey in full technical detail.

M1 Environment & Setup	M2 Data Ingestion	M3 Cleaning & Profiling	M4 Feature Engineering	M5 SQL Warehouse	M6 SSIS ETL Packages	M7 Machine Learning	M8 Prefect Orchestration	M9 Power BI Dashboard
----------------------------------	-----------------------------	-----------------------------------	----------------------------------	----------------------------	--------------------------------	-------------------------------	------------------------------------	---------------------------------

#	Milestone	Primary Deliverable	Obstacles Resolved
M 1	Environment & Project Setup	Configured dev environment, venv, Git, SQL Server, Power BI	2 — Python PATH, venv activation, two Python installs
M 2	Data Ingestion	Historical CSVs + live API fetch scripts	1 — OpenAQ returned no Cairo data → switched to WAQI
M 3	Data Cleaning & Profiling	Cleaned, normalised, validated datasets	2 — dropna() destroyed all data; date timezone mismatch
M 4	Feature Engineering	8 engineered features + health_category column	1 — Class imbalance (97% one class due to WHO thresholds)
M 5	SQL Server Data Warehouse	AetherDW_V0 with constellation schema + Python loader	2 — Connection error (08001); NULL values in all SQL columns
M 6	SSIS ETL Packages	3 SSIS packages with incremental loading	2 — Code page conflict (1252 vs 1256); package not found
M 7	Machine Learning Model	Trained Random Forest, 97% F1, saved with joblib	1 — Class imbalance causing trivial 98% accuracy
M 8	Prefect Orchestration	Automated end-to-end pipeline (~21s)	3 — utils.logger import; SSIS path; global Python in subprocess
M 9	Power BI Dashboard	Multi-page interactive dashboard with DAX measures	1 — Relationship ambiguity with multiple fact tables

4. Development Journey — Detailed Milestones

MILESTONE

1

Environment & Project Setup

Establishing the full development foundation that all subsequent work depended on

1.1 Objective

Before any data could be ingested or processed, a reliable, isolated, and reproducible development environment had to be constructed. This milestone established every tool, library, configuration, and project structure that the remaining eight milestones would build upon.

1.2 What Was Accomplished

1

Created the Aether_V0 project folder with structured subdirectories: data/raw/, data/staging/, data/processed/, pipelines/ingestion/, pipelines/transformation/, pipelines/loading/, ML/training/, ML/inference/, ML/models/, Automation/, SSIS/, Database/, Dashboard/, logs/, utils/

2

Installed and configured VS Code as the primary IDE

3

Installed and configured Git and created the GitHub repository with README.md

4

Installed SQL Server 2022 and verified connectivity via SQL Server Management Studio (SSMS)

5

Installed Power BI Desktop for dashboard development

6

Installed Visual Studio 2022 with the SQL Server Data Tools (SSDT) extension for SSIS development

7

Created a Python virtual environment (venv) to isolate all project dependencies from the global Python installation

8

Installed all core Python libraries: pandas, numpy, scikit-learn, requests, pyodbc, joblib, prefect

9

Established a strict rule: always activate venv before running any project script

1.3 Obstacles Encountered & Resolved

OBSTACLE: Python Not Recognised in PowerShell

PROBLEM

Running 'python -m venv venv' failed with: "Python was not found; run without arguments to install from Microsoft Store". After resolving that, activating the virtual

	environment with '.\venv\Scripts\Activate.ps1' produced: "running scripts is disabled on this system".
ROOT CAUSE	Two separate issues: (1) The system PATH did not include a standard Python executable — Windows had a Store alias intercepting the 'python' command. (2) PowerShell's default execution policy blocks unsigned scripts including venv activation scripts.
RESOLUTION	Step 1: Used the py launcher instead of python: 'py -m venv venv'. Step 2: Fixed PowerShell execution policy with: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser. Step 3: Activated using .\venv\Scripts\Activate.ps1. Step 4: Verified the (venv) prefix appeared in the terminal prompt.

⚠️ OBSTACLE: Two Python Installations Causing Confusion

PROBLEM	Scripts run via the VS Code Run button or via full path used the global Python 3.14 installation at C:/Users/hp/AppData/Local/Python/pythoncore-3.14-64/, which had none of the project libraries installed. pip install within the venv had no effect on scripts running under the global interpreter.
ROOT CAUSE	The machine had two Python installations: a global Python 3.14 and the project's virtual environment at D:/Aether/Aether_V0/venv/Scripts/python.exe. The VS Code Run button and subprocess.run(['python', ...]) both resolved 'python' via the system PATH, pointing to the wrong interpreter.
RESOLUTION	Established a strict rule: always activate venv first, then use 'python' command. For the Prefect automation pipeline where subprocess calls must work without manual venv activation, hardcoded the full venv Python path: PYTHON = os.path.join(ROOT, 'venv', 'Scripts', 'python.exe'). This was applied to every subprocess.run() call in the orchestration pipeline.

1.4 Milestone Outcome

COMPLETED	A fully configured, isolated development environment with all required tools installed, the project folder structure established, the Git repository initialised, and the Python virtual environment operational with all libraries. Every subsequent milestone had a stable, reproducible foundation to build on.
-----------	--

MILESTONE

2

Data Ingestion

Gathering all data sources needed by AETHER — historical archives and live API connections

2.1 Objective

Identify, access, and ingest all environmental data sources required for AETHER's training dataset and live operational pipeline. This milestone produced both the historical dataset foundation and the live API scripts used in daily automation.

2.2 Data Sources Identified & Integrated

Source	Type	Coverage	Data Acquired
Open-Meteo Historical Archive	CSV (Historical)	2000–2025	9,406 rows across 50+ meteorological columns including temperature, humidity, wind, pressure, UV index, rainfall, sunshine duration, cloud cover
Cairo Historical Pollution Dataset	CSV (Historical)	2022–2026	1,294 rows including PM2.5, PM10, ozone, nitrogen dioxide, sulphur dioxide, AQI, UV index, dust
OpenWeatherMap API	Live REST API	Daily	Current temperature, humidity, wind speed, pressure, weather description for Cairo
WAQI API (World Air Quality Index)	Live REST API	Daily	Current AQI, PM2.5, PM10, temperature, humidity, wind for Cairo station A527650

2.3 Python Fetch Scripts Created

Script	API Called	Output File	Location
fetch_weather_api.py	OpenWeatherMap Current Weather endpoint	weather_api_YYYYMMDD.json	pipelines/ingestion/
fetch_pollution_api.py	WAQI station A527650 endpoint	pollution_api_YYYYMMDD.json	pipelines/ingestion/

2.4 Obstacle Encountered & Resolved

⚠ **OBSTACLE:** OpenAQ API Returned Empty Results for Cairo

PROBLEM	The original pollution ingestion script was built using the OpenAQ v2 API with a city name parameter. When queried with city='Cairo', the API consistently returned zero results — no air quality data at all.
ROOT CAUSE	OpenAQ v2 has sparse city-name coverage for Egypt and much of Africa. The city-name based endpoint relies on partner monitoring stations that report to the OpenAQ network, and Cairo stations either don't report or are registered under different administrative names.
RESOLUTION	Switched entirely to the WAQI API (World Air Quality Index), which is purpose-built for AQI monitoring and has excellent, reliable coverage of Cairo. Used a station-specific URL: https://api.waqi.info/feed/A527650/?token=TOKEN . This station returns consistent AQI, PM2.5, PM10, and supporting weather readings for Cairo.

2.5 Milestone Outcome

COMPLETED	Two historical datasets ingested (9,406 weather rows, 1,294 pollution rows) and two live API scripts operational, each saving dated JSON files to data/raw/ on every execution. The WAQI API decision permanently replaced OpenAQ as the pollution data source.
-----------	---

MILESTONE

3

Data Cleaning & Profiling

Transforming raw, inconsistent environmental data into reliable analytics-ready datasets

3.1 Objective

Profile the raw datasets to understand their structure, quality issues, and completeness. Apply targeted cleaning transformations to produce standardised, validated, analytics-ready datasets while preserving maximum row count.

3.2 Profiling Results

Dataset	Rows	Columns	Key Issues Found
Historical Weather CSV	9,406	50+	Timezone-aware date strings incompatible with SQL Server; many irrelevant columns (soil moisture, evapotranspiration); scattered nulls across non-critical columns
Historical Pollution CSV	1,294	12	Richer pollutant coverage but smaller date range; some missing PM2.5 values; column names inconsistent with schema

3.3 Cleaning Strategy Applied

Operation	Method	Rationale
Remove duplicate rows	drop_duplicates()	Prevent inflated row counts in analytics
Drop rows with missing critical columns	dropna(subset=['date', 'temperature', 'aqi'])	Only drop rows where the measurement cannot be reconstructed — not on all 50 columns
Fill non-critical nulls	df[col].fillna(df[col].median())	Preserve row count; median is robust to outliers in weather data
Rename columns to canonical schema	df.rename(columns={'Temperature (C)': 'temperature', ...})	Ensures consistent naming across SQL loading, ML training, and SSIS packages
Normalise date format	pd.to_datetime(utc=True).dt.date	Strips timezone from date strings to produce plain YYYY-MM-DD for SQL Server compatibility
Select relevant columns	df = df[AETHER_COLUMNS]	Drops the 40+ irrelevant columns from the weather CSV

3.4 Obstacles Encountered & Resolved

⚠️ OBSTACLE: dropna() Silently Destroyed Almost All Weather Data

PROBLEM	The initial cleaning script used <code>weather_df.dropna()</code> with no arguments. The script ran without error but produced a nearly empty DataFrame — almost all 9,406 rows were silently dropped.
ROOT CAUSE	The weather CSV has 50+ columns. Pandas <code>dropna()</code> without a subset argument drops any row that has a null in ANY column. With 50+ columns, virtually every row has at least one null in some irrelevant field (e.g., soil moisture sensor at a specific depth). The result was near-total data destruction with no warning.
RESOLUTION	Changed to <code>dropna(subset=['date', 'temperature', 'humidity', 'wind'])</code> — only dropping rows where the core measurement columns are missing. Filled remaining nulls with column median: <code>df[col] = df[col].fillna(df[col].median())</code> . Result: all 9,406 weather rows and 1,294 pollution rows preserved.

⚠️ OBSTACLE: Date Conversion SQL Server Error

PROBLEM	When the cleaning script attempted to load dates into SQL Server, the operation failed with: "Conversion failed when converting date and/or time from a character string."
ROOT CAUSE	The Open-Meteo historical CSV uses timezone-aware date strings in the format '2016-04-09 00:00:00+00:00' with a UTC offset appended. SQL Server's DATE column type only accepts plain date strings in YYYY-MM-DD format and rejects timezone suffixes.
RESOLUTION	Applied <code>pd.to_datetime(df['date'], errors='coerce', utc=True).dt.date</code> — the <code>utc=True</code> flag normalises all timezone offsets to UTC, and <code>.dt.date</code> extracts the plain date portion, producing '2016-04-09' which SQL Server accepts. Added <code>errors='coerce'</code> to handle any malformed date values gracefully by converting them to NaT rather than raising an exception.

3.5 Milestone Outcome

COMPLETED	Two fully cleaned datasets: 9,406 weather rows and 1,294 pollution rows — both with consistent column names, plain YYYY-MM-DD dates, no critical nulls, and normalised data types. Data cleaning scripts saved outputs to <code>data/staging/</code> ready for feature engineering.
-----------	---

MILESTONE

4

Feature Engineering

Transforming raw measurements into meaningful environmental health intelligence

4.1 Objective

Merge the cleaned weather and pollution datasets, compute meaningful composite environmental indicators, and generate the health_category target variable. Produce the master analytical dataset that serves as input for ML training, SQL loading, and Power BI reporting.

4.2 Two-Mode Design

Feature engineering was designed to operate in two distinct modes, controlled by a command-line flag:

Mode	Trigger	Input	Output	Use Case
API Mode (default)	python feature_engineering.py	Today's API JSON files from data/raw/	One new row added to environmental_features.csv	Daily live pipeline — Prefect calls this each day
Historical Mode	python feature_engineering.py --historical	Full cleaned CSVs from data/staging/	Complete environmental_features.csv with 1,154 merged rows	ML training, initial warehouse load, recalculation runs

4.3 Date-Based Dataset Merge

In historical mode, an inner join is performed between the cleaned weather and pollution DataFrames on the date column. Only dates present in BOTH datasets survive the join. The 2022–2025 overlap between the two datasets produced 1,154 matched rows — this became the training dataset for the ML model.

KEY INSIGHT

The inner join is a deliberate design decision. Rows with weather data but no pollution data (or vice versa) cannot produce complete engineered features and would introduce training noise. The 1,154 overlap rows have complete data across all 13 training features.

4.4 Engineered Features

Feature Name	Formula	Purpose	Units
heat_index	temperature + (0.33 × humidity) – (0.70 × wind)	Perceived temperature stress combining heat and humidity while accounting for wind cooling	°C equivalent
pollution_level	(PM2.5 × 0.5) + (PM10 × 0.3) + (AQI × 0.2)	Aggregated pollution severity — weighted composite of three pollution metrics	Composite score
respiratory_stress	(PM2.5 × 0.4) + (ozone × 0.3) + (NO ₂ × 0.2) + (SO ₂ × 0.1)	Estimated respiratory health impact combining the four pollutants most directly linked to lung damage	Composite score
uv_risk	uv_index / 11 × 100, clipped 0–100	UV exposure risk normalised to a 0–100 scale where 100 = extreme UV	0–100 scale
dust_risk_index	dust + aerosol_optical_depth composite	Measures dust particle concentration and atmospheric opacity — important for Cairo's desert environment	Index
rain_wash_effect	Precipitation-based pollutant reduction estimate	Estimates how much recent rainfall reduces effective pollution load	Reduction factor
heat_stress_peak	Binary flag for extreme heat conditions	Detects days with dangerous combined heat-humidity stress	Binary (0/1)
temperature_range	Daily max – daily min temperature	Captures daily thermal variation which affects pollution behaviour	°C

4.5 Health Category Classification — Calibrated for Cairo

Environmental conditions are classified into one of five health categories using threshold-based rule logic. A critical engineering decision was to calibrate thresholds to Cairo's actual data distribution rather than WHO global guidelines.

CALIBRATION DECISION

Initial implementation used WHO global thresholds. This produced a disastrous class imbalance: 97% of Cairo's historical days fell into 'Mask Recommended Day' because Cairo's chronic pollution consistently exceeds global safe limits. Thresholds were recalibrated to Cairo's empirical distribution: AQI mean = 70, PM2.5 mean = 21 µg/m³.

Priority	Health Category	Trigger Conditions
1 (Highest)	Avoid Outdoor Activity Day	AQI > 110 OR PM2.5 > 50 µg/m³
2	Mask Recommended Day	AQI > 85 OR PM2.5 > 30 OR heat_index > 48
3	High Respiratory Risk Day	AQI > 70 OR PM2.5 > 20 OR ozone > 80 µg/m³

4	Moderate Risk Day	AQI > 55 OR PM2.5 > 14 OR NO ₂ > 30 µg/m ³
5 (Lowest)	Safe Air Day	All indicators below the above thresholds

4.6 Obstacle Encountered & Resolved

⚠️ OBSTACLE: Class Imbalance — 97% of Training Data in One Category

PROBLEM	Initial training run reported 98% accuracy but the model was completely useless: it predicted 'Mask Recommended Day' for virtually every single day regardless of actual environmental conditions. Class distribution: Mask Recommended Day = 1,127 rows (97.6%); Moderate Risk Day = 14 rows; High Respiratory Risk Day = 11 rows; Safe Air Day = 2 rows.
ROOT CAUSE	WHO global PM2.5 threshold is 15 µg/m ³ annually. Cairo's mean PM2.5 is approximately 21 µg/m ³ . Applying global thresholds to Cairo data means the vast majority of Cairo's days are classified as hazardous by WHO standards — technically correct globally but useless for building a discriminative local model. The model learned to simply predict the majority class.
RESOLUTION	Fix 1 — Analysed the actual Cairo data distribution (AQI mean=70, PM2.5 mean=21). Fix 2 — Recalibrated all thresholds in the classify_health() function to produce a balanced distribution across the 5 categories relative to Cairo's real conditions. Fix 3 — Added class_weight='balanced' to RandomForestClassifier to further compensate for any remaining imbalance. Fix 4 — Added stratify=y to train_test_split to guarantee all 5 categories appear in both train and test splits. Result: balanced class distribution and 97% accuracy that reflects genuine learning.

4.7 Milestone Outcome

COMPLETED	environmental_features.csv produced with 1,154 rows, 8 engineered features, and a balanced health_category target variable. This file serves as the master analytical dataset consumed by SQL loading, ML training, and Power BI reporting.
-----------	---

MILESTONE

5

SQL Server Data Warehouse

Designing and populating the structured analytical store for all downstream consumption

5.1 Objective

Design and implement a structured SQL Server data warehouse (AetherDW_V0) that stores processed environmental data in an analytics-optimised schema, supports Power BI reporting, and enables ML model training and inference.

5.2 Schema Design Decision — Constellation Schema

A constellation schema (also called a galaxy schema) was chosen over a simple star schema. AETHER has multiple independent fact tables — WeatherMetrics, PollutionMetrics, EnvironmentalFeatures, and RiskPredictions — that all relate to a single date dimension. This is the standard approach in production analytics platforms and was the key architectural decision that enabled clean Power BI relationships.

Table	Schema	Type	Key Columns
DimDate	dbo	Dimension	date (PK), year, month, month_name, quarter, week, day_of_week, is_weekend
WeatherMetrics	dbo	Fact	id (PK), date (FK), temperature, humidity, wind, pressure, cloud_cover, sunshine_duration, heat_index, source
PollutionMetrics	dbo	Fact	id (PK), date (FK), pm25, pm10, aqi, european_aqi, ozone, nitrogen_dioxide, sulphur_dioxide, dust, uv_index, pollution_level, source
EnvironmentalFeatures	Gold	Fact	id (PK), date (FK), all weather + pollution columns + heat_index, pollution_level, respiratory_stress, uv_risk, health_category, source
RiskPredictions	Gold	Fact	id (PK), date (FK), health_category, aqi, pm25, pollution_level, respiratory_stress, uv_risk, model_version, predicted_at
ForecastPredictions	Gold	Fact	id (PK), forecast_date, forecast_horizon, predicted_category, confidence, temperature, humidity, pm25, aqi, generated_at, model_version

5.3 Python SQL Loader (Initial Implementation)

Before SSIS packages were built, an initial Python SQL loader (load_to_sql.py) was implemented using pyodbc. This loader served as the development-phase loading mechanism and was later supplemented by SSIS for production-grade incremental loading.

5.4 Obstacles Encountered & Resolved

⚠️ OBSTACLE: SQL Server Connection Error 08001

PROBLEM	Python script using SERVER=localhost failed with: "[08001] [Microsoft][ODBC SQL Server Driver][DBNETLIB]SQL Server does not exist or access is denied." The database could not be reached at all.
ROOT CAUSE	SQL Server on Windows uses named instances rather than a simple 'localhost' address. The actual server name is the Windows machine name combined with the instance name. The connection string SERVER=localhost does not resolve to a SQL Server named instance on Windows.
RESOLUTION	Found the actual server name in the SSMS connection dialog: DESKTOP-Q5KEU1E. Updated all pyodbc connection strings to: DRIVER={SQL Server};SERVER=DESKTOP-Q5KEU1E;DATABASE=AetherDW_V0;Trusted_Connection=yes; . This resolved the connection failure immediately.

⚠️ OBSTACLE: NULL Values Inserted into Every SQL Column

PROBLEM	All rows were successfully inserted into SQL Server but every column contained NULL — temperature, humidity, AQI, and all other fields were null despite the source DataFrame containing valid data.
ROOT CAUSE	The Python insert script used row.get('temperature'), row.get('pm25'), etc. to extract values. The actual column names in the cleaned CSV were 'Temperature (C)', 'Wind Speed (km/h)', 'PM2.5 (µg/m³)' — the original raw names before schema normalisation. Because the expected canonical names did not exist in the row dictionary, row.get() returned None (None) for every field.
RESOLUTION	Added a schema normalisation step at the start of feature_engineering.py to rename all columns to canonical Aether schema names before any processing: df = df.rename(columns={'Temperature (C)': 'temperature', 'Wind Speed (km/h)': 'wind', ...}). Also added a val() helper function in load_to_sql.py to convert pandas NaN values to Python None: pandas NaN is not the same as Python None; pyodbc requires None to insert SQL NULLs.

5.5 Milestone Outcome

COMPLETED	AetherDW_V0 fully designed with constellation schema, DDL scripts written and executed, all 6 tables created with proper constraints and relationships, and the Python loader successfully populating EnvironmentalFeatures with 1,154 rows from the historical dataset.
-----------	--

MILESTONE

6

SSIS ETL Packages*Upgrading to production-grade incremental loading with SQL Server Integration Services*

6.1 Objective

Replace the development-phase Python SQL loader with professional SSIS ETL packages that implement incremental loading (insert new rows only, never duplicates), handle data type conversions, and support safe re-execution.

6.2 Tool Setup

Component	Version / Configuration	Purpose
Visual Studio 2022	With SQL Server Integration Services Projects 2022 extension	SSIS package development IDE
SSIS Project	AetherSSIS — saved at D:/Aether/Aether_V0/SSIS/AetherSSIS/	Container for all SSIS packages
dtexec.exe	SQL Server 2022 SSIS runtime	Command-line SSIS package executor — called by Prefect pipeline

6.3 SSIS Packages Created

Package	Source File	Destination Table
LoadEnvironmentalFeatures.dtsx	data/processed/environmental_features.csv	Gold.EnvironmentalFeatures
LoadWeatherMetrics.dtsx	data/staging/cleaned_weather.csv	dbo.WeatherMetrics
LoadPollutionMetrics.dtsx	data/staging/cleaned_pollution.csv	dbo.PollutionMetrics

6.4 Incremental Loading Design

Rather than truncating and reloading tables on each run (a naive but common approach), every SSIS package implements incremental loading using a Lookup component:

Component	Configuration	Behaviour
-----------	---------------	-----------

Flat File Source	Reads CSV with code page 1252; all columns initially read as strings	Ingests every row from the source file
Data Conversion	Converts string columns to correct SQL types: DT_DBDATE for dates, DT_R4 for floats, DT_STR for categories	Type-safe data before loading into SQL Server
Lookup	Checks if the incoming date already exists in the destination SQL table	Separates new rows from existing rows
Match Output	No destination connected — rows with matching dates are discarded	Existing dates are silently skipped; no duplicates
No Match Output	Connected to OLE DB Destination	Only genuinely new dates are inserted
DESIGN PRINCIPLE	This incremental approach means every SSIS package can be re-run any number of times safely. If the pipeline fails mid-way and restarts, no duplicate rows accumulate. This is a professional-grade ETL pattern as opposed to the truncate-and-reload approach.	

6.5 Obstacles Encountered & Resolved

⚠️ **OBSTACLE: SSIS Code Page Conflict Error (1252 vs 1256)**

PROBLEM	SSIS package validation failed with: "Column 'health_category' cannot be processed because more than one code page (1252 and 1256) are specified for it." The package refused to execute.
ROOT CAUSE	The development machine runs Windows with an Arabic locale (Egypt), which uses Windows code page 1256 for text rendering. SSIS inherits the system code page for string processing (1256). The source CSV file was generated by Python using UTF-8 / Latin-1 (code page 1252). The mismatch caused SSIS to be unable to process the string column 'health_category' consistently.
RESOLUTION	In the Flat File Connection Manager properties, explicitly changed the Code Page setting from the auto-detected 1256 to 1252. In the Data Conversion component for the health_category column, explicitly set the Code Page to 1252. This told SSIS to treat the entire text file as English/Latin encoding throughout — consistent with how Python generated it.

⚠️ **OBSTACLE: SSIS Package Not Found at Runtime**

PROBLEM	When Prefect called dtexec.exe to run the SSIS package, dtexec returned: "Unable to load the package as XML — the file cannot be opened."
----------------	---

ROOT CAUSE	Visual Studio saved the compiled .dtsx file to SSIS/AetherSSIS/ (the Visual Studio project subfolder). The Prefect pipeline script pointed to ssis/ (a different top-level folder used as the deployment location). The two paths were different.
RESOLUTION	Copied the compiled .dtsx file to the expected deployment location: Copy-Item 'SSIS/AetherSSIS/LoadEnvironmentalFeatures.dtsx' 'ssis/'. Updated the SSIS_PACKAGE constant in prefect_pipeline.py to use the correct path. Going forward, established a convention of copying the final .dtsx to ssis/ after every Visual Studio build.

6.6 Milestone Outcome

COMPLETED	Three production SSIS packages operational with incremental loading logic. Each package can be executed safely multiple times with no duplicate rows. All three packages called from Prefect via dtexec.exe as part of the automated daily pipeline.
-----------	--

MILESTONE
7

Machine Learning Model

Building the intelligence core that classifies environmental conditions into health risk categories

7.1 Objective

Train a supervised classification model that learns the relationship between environmental conditions and health risk categories from three years of Cairo historical data. The model must produce meaningful predictions for all five health categories, not just the majority class.

7.2 Model Architecture

Component	Choice	Rationale
Problem Type	Multi-class Classification	5 target categories (health_category)
Algorithm	Scikit-learn RandomForestClassifier	Handles tabular data well; manages non-linear relationships; robust to mixed feature scales; resists overfitting; no feature normalisation required
n_estimators	200	Sufficient trees for stable variance reduction without excessive compute
class_weight	'balanced'	Critical: automatically weights minority classes higher to compensate for any remaining class imbalance after threshold recalibration
Train/Test Split	80/20 with stratify=y	stratify ensures all 5 categories appear in both train and test sets even if some are small
Model Persistence	joblib (.pkl file)	Serialises trained model for reuse in inference and forecast scripts without retraining

7.3 Training Features (13 Total)

Feature Group	Features
Weather (7)	temperature, humidity, wind, pressure, precipitation, uv_index, sunshine_duration
Pollution (6 core)	pm25, pm10, aqi, ozone, nitrogen_dioxide, sulphur_dioxide
Engineered (5 used)	heat_index, pollution_level, respiratory_stress, uv_risk, pressure (shared)

7.4 Model Performance Results

After Cairo-specific threshold recalibration (see Milestone 4), the model achieved the following classification performance on the held-out test set:

97% Overall Accuracy <i>Holdout test set</i>	1.00 Avoid Outdoor F1 <i>Perfect classification</i>	0.97 High Resp. Risk F1	0.95 Safe Air Day F1 <i>Hardest category</i>
---	--	-----------------------------------	---

Health Category	F1 Score	Precision	Recall	Interpretation
Avoid Outdoor Activity Day	1.00	1.00	1.00	Extreme conditions are perfectly distinguishable
High Respiratory Risk Day	0.97	0.97	0.97	Strong signal from PM2.5 + ozone combination
Mask Recommended Day	0.97	0.96	0.98	Core daily risk category — very reliable
Moderate Risk Day	0.96	0.97	0.95	Boundary cases between safe and risky
Safe Air Day	0.95	0.95	0.96	Rarest in Cairo — model still learns it well
Overall Accuracy	97%	—	—	Meaningful accuracy after class balancing

7.5 Feature Importance Analysis

Rank	Feature	Importance	Insight
1	AQI (US)	31.3%	Dominant predictor — the single most informative signal for health classification
2	PM2.5	21.3%	Core fine particulate measurement — direct lung health indicator
3	pollution_level (engineered)	14.4%	Validates feature engineering investment — the composite index adds real predictive signal beyond raw metrics
4	Ozone	7.7%	Extended pollutant data contributes meaningfully — justifies multi-source data collection
5	PM10	6.5%	Secondary particulate signal complements PM2.5

6–13	Other features	19.8%	Temperature, humidity, wind and remaining features contribute collectively
------	----------------	-------	--

7.6 Obstacle Encountered & Resolved

⚠️ OBSTACLE: Trivial 98% Accuracy Due to Class Imbalance	
PROBLEM	First training run reported 98% accuracy on the test set. However, examining the confusion matrix revealed the model predicted 'Mask Recommended Day' for every single test sample regardless of actual conditions. Precision and recall for all other categories were 0.00.
ROOT CAUSE	This obstacle was a consequence of the class imbalance created in Milestone 4 before threshold recalibration was applied. With 1,127 out of 1,154 rows (97.6%) labelled as 'Mask Recommended Day', the model could achieve 97.6% accuracy by predicting that one class universally — a common failure mode in imbalanced classification.
RESOLUTION	This obstacle was the direct motivation for the threshold recalibration performed in Milestone 4. Once thresholds were recalibrated to Cairo's actual pollution distribution, the class balance was restored across all 5 categories. <code>class_weight='balanced'</code> was added to <code>RandomForestClassifier</code> as an additional safeguard. The model was retrained on the rebalanced dataset, producing 97% meaningful accuracy.

7.7 Milestone Outcome

COMPLETED	Trained Random Forest model with 97% F1 score saved as <code>ML/models/environmental_risk_model.pkl</code> via <code>joblib</code> . Daily inference script <code>predict_risk.py</code> operational — loads the saved model, accepts today's environmental features, generates a <code>health_category</code> prediction, and inserts the result into <code>Gold.RiskPredictions</code> .
-----------	--

MILESTONE

8

Prefect Orchestration

Automating the entire environmental intelligence lifecycle into a single daily pipeline

8.1 Objective

Eliminate all manual execution steps by orchestrating the complete pipeline — API fetching, feature engineering, SQL loading, and ML prediction — into a single automated daily workflow with retry logic, structured logging, and execution monitoring.

8.2 Tool Evaluation & Selection

Tool	Evaluated?	Decision	Reason
Apache Airflow	Yes — attempted	Rejected	os.register_at_fork() is not available on Windows — Airflow's worker process model fundamentally requires POSIX fork semantics. Could not be made to run stably on a 4 GB RAM Windows machine.
Windows Task Scheduler	Yes — considered	Rejected	Too basic — no task-level retry logic, no structured logging, no execution monitoring, no dependency chaining between steps.
Prefect	Yes — selected	Adopted	Windows-native, Python-first, lightweight (runs in-process), professional retry and logging capabilities, minimal resource overhead.

8.3 Pipeline Architecture

Task Function	Calls	Output	Retry Config
fetch_weather()	fetch_weather_api.py	weather_api_YYYYMMDD.json saved to data/raw/	retries=2, delay=10s
fetch_pollution()	fetch_pollution_api.py	pollution_api_YYYYMMDD.json saved to data/raw/	retries=2, delay=10s
feature_engineering()	feature_engineering.py (API mode)	environmental_features.csv updated with today's row	retries=2, delay=10s
load_ssis()	dtexec.exe → LoadEnvironmentalFeatures.dtsx	New row inserted into Gold.EnvironmentalFeatures (if date is new)	retries=1, delay=15s
run_predictions()	predict_risk.py	New row inserted into Gold.RiskPredictions	retries=2, delay=10s

8.4 Key Engineering Decisions

Decision	Implementation	Why
Python path management	<code>PYTHON = os.path.join(ROOT, 'venv', 'Scripts', 'python.exe')</code> — hardcoded absolute path in all <code>subprocess.run()</code> calls	Prevents <code>subprocess.run(['python', ...])</code> from picking up the global Python installation via <code>PATH</code>
Working directory	<code>cwd=ROOT</code> set in all <code>subprocess.run()</code> calls	Ensures all relative file paths inside called scripts resolve correctly regardless of where Prefect launches from
Logging	Python's built-in logging module → <code>logs/pipeline.log</code> (not Prefect's own logger)	Centralised, structured log with timestamps, execution times, row counts, and failures in one file
Execution timing	<code>time.time()</code> before and after each subprocess call	Each script's execution time appears in <code>pipeline.log</code> for performance monitoring
stderr handling	<code>stdout</code> and <code>stderr</code> captured separately; <code>stderr</code> logged as <code>WARNING</code> even on success	Some tools write informational messages to <code>stderr</code> — treating all <code>stderr</code> as <code>ERROR</code> creates false alarms

8.5 Obstacles Encountered & Resolved

⚠️ OBSTACLE: ModuleNotFoundError: No module named 'utils.logger'

PROBLEM	<code>prefect_pipeline.py</code> attempted <code>'from utils.logger import setup_logger'</code> at the top of the file. Python raised <code>ModuleNotFoundError</code> when Prefect executed the script.
ROOT CAUSE	Python treats the script's containing folder as the root for module resolution. <code>prefect_pipeline.py</code> lives in <code>Automation/</code> . Python looked for a <code>utils/</code> subfolder inside <code>Automation/</code> — which doesn't exist. The <code>utils/</code> folder is a sibling of <code>Automation/</code> at the project root level.
RESOLUTION	Eliminated the external <code>utils.logger</code> dependency entirely. Moved all logging setup code directly inline into <code>prefect_pipeline.py</code> using Python's built-in logging module. Also added <code>sys.path.insert(0, ROOT)</code> at the top of the script as a safeguard for any future imports. Result: zero import dependencies between pipeline files.

⚠️ OBSTACLE: SSIS Package Not Found by Prefect (dtexec Path Mismatch)

PROBLEM	The <code>load_ssis()</code> task called <code>dtexec.exe</code> with a path to the <code>.dtsx</code> file. <code>dtexec</code> returned: "Unable to load the package as XML — the file cannot be opened." The task failed on every run.
ROOT CAUSE	The <code>SSIS_PACKAGE</code> constant in <code>prefect_pipeline.py</code> pointed to <code>ssis/LoadEnvironmentalFeatures.dtsx</code> (the intended deployment folder). The actual <code>.dtsx</code> file was saved by Visual Studio to <code>SSIS/AetherSSIS/LoadEnvironmentalFeatures.dtsx</code> . The two folder paths did not match.
RESOLUTION	Copied the compiled <code>.dtsx</code> to the expected path: Copy-Item 'SSIS/AetherSSIS/LoadEnvironmentalFeatures.dtsx' 'ssis/'. Updated the <code>SSIS_PACKAGE</code> constant

to the correct absolute path. Documented the deployment convention: always copy .dtsx files from the Visual Studio project folder to ssis/ after each build.

⚠️ **OBSTACLE: Prefect Tasks Running Under Global Python Instead of venv**

PROBLEM	Even with the venv activated in the terminal, Prefect's subprocess.run(['python', script]) calls resolved to the global Python 3.14 installation. Scripts failed with ModuleNotFoundError for pandas, pyodbc, and other venv-only libraries.
ROOT CAUSE	subprocess.run(['python', ...]) resolves 'python' using the system PATH environment variable. On this machine, the global Python 3.14 installation appeared on PATH before the venv's Python. Activating venv in the interactive terminal modifies PATH for that terminal session, but subprocess calls inherit the parent process's environment — which may not include the venv PATH modification consistently.
RESOLUTION	Replaced all subprocess.run(['python', script]) calls with subprocess.run([PYTHON, script]) where PYTHON is the hardcoded absolute path to the venv's python.exe: PYTHON = os.path.join(ROOT, 'venv', 'Scripts', 'python.exe'). This guarantees every subprocess call uses the project's virtual environment regardless of the PATH state.

8.6 Validated Pipeline Execution Log

After all obstacles were resolved, the complete pipeline was validated end-to-end. Actual Prefect execution output:

Timestamp	Event	State
05:10:25	Flow run 'nebulous-hippogriff' — Beginning flow run for 'Aether Pipeline'	Started
05:10:28	Task run 'fetch_weather-744' — fetch_weather_api.py executed (~3s)	Completed
05:10:30	Task run 'fetch_pollution-209' — fetch_pollution_api.py executed (~2s)	Completed
05:10:32	Task run 'feature_engineering-4e6' — feature_engineering.py executed (~2s)	Completed
05:10:34	Task run 'load_ssis-641' — dtexec LoadEnvironmentalFeatures.dtsx executed (~12s)	Completed
05:10:46	Task run 'run_predictions-db0' — predict_risk.py executed (~1s)	Completed
05:10:46	Flow run 'nebulous-hippogriff' — Finished in state Completed()	Total: ~21 seconds

8.7 Milestone Outcome

COMPLETED	Full Prefect orchestration pipeline operational. The entire AETHER data pipeline — from live API fetch through ML prediction — executes automatically in approximately 21 seconds daily with retry logic, structured logging, and zero manual intervention.
------------------	---

MILESTONE

9

Power BI Dashboard

Delivering environmental health intelligence through interactive analytical reporting

9.1 Objective

Transform all processed data, ML predictions, and forecasts into an interactive, executive-grade Power BI dashboard that makes environmental health intelligence accessible to both technical and non-technical audiences.

9.2 Data Model Design

The Power BI model mirrors the SQL Server constellation schema with DimDate as the central filtering dimension. Most visuals draw from EnvironmentalFeatures directly since it contains all merged weather, pollution, and engineered features in one table.

Relationship	Type	Filter Direction	Purpose
DimDate[Date] → EnvironmentalFeatures[date]	One-to-Many	Single (DimDate → Fact)	Enables date slicing across main analytics
DimDate[Date] → RiskPredictions[date]	One-to-Many	Single (DimDate → Fact)	Synchronises ML prediction view with date filter
DimDate[Date] → ForecastPredictions[forecast_date]	One-to-Many	Single (DimDate → Fact)	Aligns forecast view to date dimension
DimDate[Date] → WeatherMetrics[date]	One-to-Many	Single	Weather detail page filtering
DimDate[Date] → PollutionMetrics[date]	One-to-Many	Single	Pollution detail page filtering

DESIGN DECISION

Single-direction filtering was used for all relationships. Bidirectional cross-filtering between multiple fact tables through a shared dimension creates ambiguous filter paths that cause incorrect aggregations and slow performance. Single-direction is the industry-standard approach for constellation schemas in Power BI.

9.3 DAX Measures

Measure Name	DAX Formula	Used In
Total Days	COUNTROWS(EnvironmentalFeatures)	KPI cards, rate denominators
Average AQI	AVERAGE(EnvironmentalFeatures[aqi])	Environmental overview KPI card
Dangerous Days Rate	DIVIDE([Avoid Outdoor Days] + [Mask Recommended Days], [Total Days])	Executive health risk summary
WHO PM2.5 Exceedance Rate	DIVIDE(CALCULATE(COUNTROWS(EnvironmentalFeatures), EnvironmentalFeatures[pm25]>15), [Total Days])	WHO compliance monitoring visual
Latest Health Category	CALCULATE(LASTNONBLANK(RiskPredictions[health_category],1), REMOVEFILTERS(DimDate))	Home page current status card — always shows today regardless of date filter
Avoid Outdoor Days	CALCULATE([Total Days], EnvironmentalFeatures[health_category]="Avoid Outdoor Activity Day")	Category distribution chart
Mask Recommended Days	CALCULATE([Total Days], EnvironmentalFeatures[health_category]="Mask Recommended Day")	Category distribution chart

9.4 Dashboard Pages

Page	Purpose	Key Visuals
Home	Executive overview — instant status at a glance	Current AQI card, current health category, heat index, PM2.5 level, dynamic health recommendation text (DAX-generated), navigation buttons
Environmental Overview	Historical environmental analytics	AQI trend line chart, heat index trend, health category donut chart, temperature vs pollution scatter, monthly pollution bar chart, date slicer
Health Intelligence	AI prediction analysis and respiratory risk	Health status recommendation card, stacked monthly category trends, prediction history table, respiratory stress scatter, 3-day forecast cards
Forecast Analytics	Future environmental outlook	Day +1/+2/+3 predicted category cards, AQI forecast trend, forecast confidence bar, model version tracking
Seasonal Analysis	Recurring seasonal pollution patterns	Winter spike detection, summer heat stress heatmap, dust storm period identification, wind dispersal effect analysis

9.5 Obstacle Encountered & Resolved

⚠️ OBSTACLE: Power BI Relationship Ambiguity Across Multiple Fact Tables	
PROBLEM	After importing all tables and creating relationships, Power BI generated multiple warnings about ambiguous filter paths. Several visuals produced incorrect aggregations — AQI values were showing in the prediction history table and prediction labels were appearing in the raw environmental chart.
ROOT CAUSE	The initial model attempted bidirectional cross-filtering between fact tables through DimDate. With multiple fact tables (EnvironmentalFeatures, RiskPredictions, WeatherMetrics, PollutionMetrics) all connected bidirectionally to DimDate, Power BI's DAX engine faced multiple paths to calculate any filtered measure — an ambiguous filter context problem inherent to galaxy schemas with bidirectional relationships.
RESOLUTION	Redesigned all relationships to use single-direction filtering only (DimDate filters fact tables, not vice versa). This matches the constellation schema pattern where the dimension drives filtering downward into facts. All visuals were reconstructed after the model redesign. The ambiguity warnings disappeared and all aggregations produced correct results.

9.6 Milestone Outcome

COMPLETED	Full multi-page Power BI dashboard operational with 5 pages, all DAX measures correctly calculated, constellation schema with single-direction filtering, conditional formatting on health category cards, and dynamic health recommendations generated by DAX logic. Dashboard file saved as Dashboard/Aether_V0_Dashboard.pbix.
-----------	---

5. Key Performance Indicators

The following KPIs were defined at project initiation to measure successful delivery across all system components. All targets were met or exceeded.

5.1 Data Engineering KPIs

KPI	Target	Achieved	Status
Missing Data Handling	100%	100% — dropna(subset=) + median imputation	Met
Duplicate Data Removal	100%	100% — drop_duplicates() at every stage	Met
Data Cleaning Runtime	< 2 minutes	< 30 seconds for full historical pipeline	Exceeded
Data Quality Score	≥ 95%	All critical columns fully populated	Met
Historical Data Coverage	≥ 1,000 rows	1,154 merged rows (2022–2025)	Met

5.2 SQL Warehouse KPIs

KPI	Target	Achieved	Status
Query Execution Time	< 2 seconds	< 1 second on all tested queries	Exceeded
Database Consistency	100%	Constellation schema with FK constraints	Met
Environmental Indicators per Row	≥ 5	17 columns in EnvironmentalFeatures	Exceeded
Incremental Loading Safety	No duplicates on re-run	SSIS Lookup component prevents all duplicates	Met

5.3 Machine Learning KPIs

KPI	Target	Achieved	Status
Overall Prediction Accuracy	80–90%	97% overall accuracy	Exceeded
F1 Score per Category	≥ 0.80	0.95–1.00 across all 5 categories	Exceeded

Risk Prediction Lead Time	24 hours	Day +1, +2, +3 forecasts generated	Exceeded
Trend Detection Accuracy	≥ 90%	Validated on holdout test set	Met
Class Balance	All categories represented	Balanced after Cairo calibration	Met

5.4 Visualisation & Automation KPIs

KPI	Target	Achieved	Status
Dashboard Load Time	< 3 seconds	< 2 seconds observed	Met
Visualisation Coverage	≥ 90% of key metrics	100% of defined metrics visualised	Exceeded
Pipeline Automation	Zero manual steps daily	Full automation via Prefect (~21s)	Met
Pipeline Reliability	Retry on failure	@task(retries=2) on all 5 tasks	Met

6. Lessons Learned

The following engineering insights were gained through direct experience across the nine development milestones. Each represents a principle that shaped the project's design decisions.

Lesson	Experience	Principle Applied
Data quality is the hardest part	The dropna() issue alone would have silently destroyed the entire 9,406-row dataset without any error message. More time was spent cleaning and validating data than on any other single milestone.	Always verify row counts before and after every transformation step. Defensive data engineering means being suspicious of every operation, not just the ones that raise exceptions.
Execution environment matters more than code	The single most recurring obstacle across the entire project was running code in the wrong Python environment — affecting the initial setup, SSIS paths, and Prefect subprocesses across three different milestones.	Understanding the difference between global Python, virtual environments, and subprocess execution contexts is foundational. Every production script must have its interpreter path determined deterministically, not via PATH.
Domain knowledge shapes model quality	The ML model went from 98% trivially useless accuracy to 97% genuinely meaningful accuracy by understanding Cairo's pollution reality. Applying WHO global thresholds to Cairo data created a model that learned nothing.	Calibrate thresholds to actual data distribution, not global standards. A model trained on incorrectly labelled data cannot be fixed by tuning hyperparameters — fix the labels first.
Tool selection should match real constraints	Apache Airflow is an excellent orchestration platform for Linux-based production environments. It is not appropriate for a 4 GB RAM Windows development machine where os.register_at_fork() is unavailable.	Professional engineering means choosing the right tool for the actual context, not the most impressive-sounding option. Prefect delivered every required capability with a fraction of the overhead.
Schema design prevents downstream problems	Every canonical column naming decision made in feature_engineering.py had consequences in SQL loading, SSIS package mapping, ML feature selection, and Power BI column references. Inconsistent naming created the NULL-in-all-columns obstacle.	Invest in a consistent, documented schema from the start. A rename() map in one script that propagates canonical names everywhere is worth more than any individual feature.
Constellation over simple star	Initially designed a simple star schema but encountered relationship ambiguity when adding multiple fact tables to Power BI. The ambiguous filter paths caused incorrect aggregations.	Galaxy/constellation schema with single-direction filtering is the correct pattern when multiple fact tables share a dimension. Bidirectional cross-filtering through a shared dimension creates ambiguous DAX filter contexts in Power BI.

7. Future Enhancements

Priority	Enhancement	Description	Technology Stack
High	Real-Time Streaming	Replace daily batch ingestion with live AQI streaming and continuous dashboard updates	Apache Kafka / Azure Event Hubs, Azure Stream Analytics
High	Cloud Migration	Deploy entire pipeline, warehouse, and ML system to managed cloud infrastructure	Microsoft Azure Data Factory, Azure SQL, Azure ML
High	Automated Model Retraining	Schedule periodic model retraining as new data accumulates to prevent model drift	MLflow, Azure Machine Learning, automated pipeline trigger
Medium	Geospatial Analytics	Add district-level pollution mapping and spatial clustering for Cairo neighbourhoods	Azure Maps, GeoPandas, PostGIS
Medium	Deep Learning Forecasting	Experiment with LSTM and Transformer-based models for improved multi-day forecasting	TensorFlow / PyTorch, time-series forecasting architectures
Medium	REST API Serving Layer	Expose ML predictions and current health status via a public REST API for third-party integration	FastAPI, Azure Functions, API Management
Low	Mobile Dashboard	Citizen-facing mobile application with push notifications for high-risk day alerts	React Native, Power Apps, Azure Logic Apps
Low	IoT Sensor Integration	Connect physical air quality sensors for hyper-local pollution monitoring beyond API data	Azure IoT Hub, MQTT protocol, Raspberry Pi sensors

8. Project Deliverables

8.1 Technical Deliverables

Deliverable	Description	Location	Format
ETL Ingestion Scripts	fetch_weather_api.py, fetch_pollution_api.py — daily live data fetch from two APIs	pipelines/ingesti on/	Python (.py)
Data Cleaning Script	data_cleaning.py — profiling, null handling, normalisation, column standardisation	pipelines/transfo rmation/	Python (.py)
Feature Engineering Script	feature_engineering.py — dual-mode (API/historical), 8 engineered features, health classification	pipelines/transfo rmation/	Python (.py)
SSIS ETL Packages	LoadEnvironmentalFeatures.dtsx, LoadWeatherMetrics.dtsx, LoadPollutionMetrics.dtsx	ssis/	.dtsx
SQL Server Warehouse DDL	AetherDW_V0 — full DDL with 6 tables, constraints, and indexes	Database/DDL.sq l	SQL (.sql)
ML Training Script	train_risk_model.py — Random Forest training with calibrated thresholds	ML/training/	Python (.py)
Trained ML Model	environmental_risk_model.pkl — serialised Random Forest, 97% F1 score	ML/models/	joblib (.pkl)
ML Inference Script	predict_risk.py — daily prediction generation and SQL insertion	ML/inference/	Python (.py)
Prefect Pipeline	prefect_pipeline.py — full orchestration flow with 5 tasks, retry logic, logging	Automation/	Python (.py)
Power BI Dashboard	Aether_V0_Dashboard.pbix — 5-page interactive dashboard	Dashboard/	.pbix
Pipeline Log	pipeline.log — timestamped execution trace with row counts and timings	logs/	.log
Technical Documentation	This document — full development journey	root/	DOCX / PDF

8.2 Repository Structure

Folder / File	Contents
Automation/prefect_pipeline.py	Prefect orchestration flow — 5 tasks, retry logic, structured logging
Dashboard/Aether_V0_Dashboard.pbix	Power BI desktop dashboard — 5 pages, constellation schema, DAX measures
data/raw/	Daily API JSON files (weather_api_YYYYMMDD.json, pollution_api_YYYYMMDD.json) + original CSVs
data/staging/	Cleaned CSV outputs from data_cleaning.py
data/processed/environmental_features.csv	Merged, engineered master dataset (1,154 rows for historical; grows daily)
Database/DDL.sql	Full SQL Server warehouse DDL — all 6 tables with constraints
Database/SQLQuery1.sql	Validation queries for data quality checks
logs/pipeline.log	Structured execution log — timestamps, row counts, failures
ML/inference/predict_risk.py	Daily ML inference script
ML/models/environmental_risk_model.pkl	Serialised trained Random Forest model
ML/training/train_risk_model.py	Model training script with Cairo-calibrated thresholds
pipelines/ingestion/	fetch_weather_api.py, fetch_pollution_api.py
pipelines/transformation/	data_cleaning.py, feature_engineering.py
SSIS/AetherSSIS/	Visual Studio SSIS project source
ssis/LoadEnvironmentalFeatures.dtsx	Deployed SSIS package — incremental loading with Lookup deduplication
requirements.txt	Python dependency list for venv setup
README.md	Project overview, setup instructions, architecture summary

9. Conclusion

AETHER demonstrates what becomes possible when data engineering, automation, machine learning, and business intelligence are integrated around a clearly defined real-world problem. The platform successfully bridges the gap between raw environmental monitoring data and the actionable public health intelligence that Cairo's citizens, healthcare providers, and policymakers need.

Nine milestones were completed, fifteen technical obstacles were encountered and resolved, and every project KPI was met or exceeded. The journey from a blank project folder to a fully automated environmental intelligence system that runs end-to-end in 21 seconds documents not just the final product, but the engineering decisions, dead ends, and hard-won solutions that shaped it.

What Was Built

9 Milestones All completed	15 Obstacles All resolved	97% ML Accuracy F1 score	21s Daily Pipeline Full automation	6 DB Tables Constellation schema
---	--	---	---	---

AETHER architecture was designed from the beginning with future scalability in mind. Every component — the medallion storage layers, the constellation schema, the Prefect orchestration, the joblib-serialised model — is a stepping stone toward a cloud-deployed, real-time streaming, mobile-accessible environmental health platform.